

SWEN90010 - High Integrity Systems Engineering

Hoare Logic

Christine Rizkallah



christine.rizkallah@unimelb.edu.au



<http://people.eng.unimelb.edu.au/rizkallahc>



HOARE LOGIC (PROVING PROGRAMS CORRECT)

Course Overview - Revisited



THE UNIVERSITY OF
MELBOURNE

Safety and Security Engineering

Formal Specifications

Validation of
Formal Specifications

Reasoning about
Correctness of Programs

Design of
Fault Tolerant Systems

Course Overview - Revisited



THE UNIVERSITY OF
MELBOURNE

Safety and Security Engineering

Formal Specifications

Validation of
Formal Specifications

Reasoning about
Correctness of Programs

Design of
Fault Tolerant Systems

Reasoning about
Correctness of Programs

Lectures:

- Hoare Logic for Programs
- Loop Invariants
- Pointers
- Separation Logic



HOARE LOGIC (PROVING PROGRAMS CORRECT)

1. State a Theorem about Correctness of a Program
2. Proof the Theorem using Hoare Logic

HOARE LOGIC (PROVING PROGRAMS CORRECT)

1. State a Theorem about Correctness of a Program
2. Proof the Theorem using Hoare Logic

≡ Pythagorean theorem

✓ Proofs using constructed squares

Rearrangement proofs

Algebraic proofs

✓ Other proofs of the theorem

Proof using similar triangles

Einstein's proof by dissection without rearrangement

Euclid's proof

Proofs by dissection and rearrangement

Proof by area-preserving shearing

Other algebraic proofs

Proof using differentials

From Wikinedia

Preview of Proofs about Programs



THE UNIVERSITY OF
MELBOURNE

Structural Proofs

- Proof tree represents logical structure of the program

$$\frac{\frac{\frac{\overline{A}}{1} \quad A \rightarrow B}{B} \quad B \rightarrow C}{\frac{C}{A \rightarrow C} 1}$$

Caveats



THE UNIVERSITY OF
MELBOURNE

If I prove my program is **correct**,
does that guarantee it will have no **bugs**?

Caveats

If I prove my program is **correct**,
does that guarantee it will have no **bugs**?

No

Caveats

If I prove my program is **correct**,
does that guarantee it will have no **bugs**?

No

Incorrect Specification
(you proved the wrong thing)

Caveats



THE UNIVERSITY OF
MELBOURNE

If I prove my program is **correct**,
does that guarantee it will have no **bugs**?

No

Incorrect Specification
(you proved the wrong thing)

```
method Find(xs: seq<int>, x: int, start: int, end: int) returns (i: int)
  requires start < |xs|
  requires end < |xs|
  ensures start < i < end
```

Caveats



THE UNIVERSITY OF
MELBOURNE

If I prove my program is **correct**,
does that guarantee it will have no **bugs**?

No

Incorrect Specification
(you proved the wrong thing)

Under Specification
(you did not specify the complete specifications)

```
data List a = Nil
            | Cons a (List a)
```

Caveats

If I prove my program is **correct**,
does that guarantee it will have no **bugs**?

No

Incorrect Specification
(you proved the wrong thing)

Under Specification
(you did not specify the complete specifications)

Program differs from what was proved
(proofs are always over mathematical models and states)

Caveats

If I prove my program is **correct**,
does that guarantee it will have no **bugs**?

No

Incorrect Specification
(you proved the wrong thing)

Under Specification
(you did not specify the complete specifications)

Program differs from what was proved
(proofs are always over mathematical models and states)

Program's execution differs from ideal
(compiler or hardware error, hardware failure etc)

Misconception



Proving is difficult

(Not if you understand programming carefully)

Proving using Hoare Logic is fairly mechanical

Programming Language



THE UNIVERSITY OF
MELBOURNE

Expressions:

Booleans

Statements

Procedures:

Programming Language



THE UNIVERSITY OF
MELBOURNE

Expressions:

$$E \quad := \quad NE \mid a[NE]$$
$$NE \quad := \quad n \mid v \mid E_1 + E_2 \mid E_1 - E_2 \mid E_1 \times E_2 \mid E_1 / E_2 \mid \text{etc.} \dots$$

where n is a number, v is a variable, a is an array variable, NE is a numerical expression, and E an expression.

Booleans

Statements

Procedures:

Programming Language



THE UNIVERSITY OF
MELBOURNE

Expressions:

$$E \quad := \quad NE \mid a[NE]$$
$$NE \quad := \quad n \mid v \mid E_1 + E_2 \mid E_1 - E_2 \mid E_1 \times E_2 \mid E_1 / E_2 \mid \text{etc.} \dots$$

where n is a number, v is a variable, a is an array variable, NE is a numerical expression, and E an expression.

Booleans

$$B \quad := \quad \text{true} \mid \text{false} \mid \neg B \mid E_1 = E_2 \mid E_1 \leq E_2 \mid \text{etc.} \dots$$

Statements

Procedures:

Programming Language



THE UNIVERSITY OF
MELBOURNE

Expressions:

$$E \quad := \quad NE \mid a[NE]$$
$$NE \quad := \quad n \mid v \mid E_1 + E_2 \mid E_1 - E_2 \mid E_1 \times E_2 \mid E_1 / E_2 \mid \text{etc.} \dots$$

where n is a number, v is a variable, a is an array variable, NE is a numerical expression, and E an expression.

Booleans

$$B \quad := \quad \text{true} \mid \text{false} \mid \neg B \mid E_1 = E_2 \mid E_1 \leq E_2 \mid \text{etc.} \dots$$

Statements

$$S \quad := \quad \text{skip} \mid v := E \mid a[NE] := E \mid p(E_1, \dots, E_n) \mid S_1; S_2 \mid \\ \text{if } B \text{ then } S_1 \text{ else } S_2 \text{ endif} \mid \text{while } B \text{ do } C \text{ done}$$

where v is a variable, a is an array variable, and p is a procedure name.

Procedures:

Programming Language



THE UNIVERSITY OF
MELBOURNE

Expressions:

$$E \quad := \quad NE \mid a[NE]$$
$$NE \quad := \quad n \mid v \mid E_1 + E_2 \mid E_1 - E_2 \mid E_1 \times E_2 \mid E_1 / E_2 \mid \text{etc.} \dots$$

where n is a number, v is a variable, a is an array variable, NE is a numerical expression, and E an expression.

Booleans

$$B \quad := \quad \text{true} \mid \text{false} \mid \neg B \mid E_1 = E_2 \mid E_1 \leq E_2 \mid \text{etc.} \dots$$

Statements

$$S \quad := \quad \text{skip} \mid v := E \mid a[NE] := E \mid p(E_1, \dots, E_n) \mid S_1; S_2 \mid \\ \text{if } B \text{ then } S_1 \text{ else } S_2 \text{ endif} \mid \text{while } B \text{ do } C \text{ done}$$

where v is a variable, a is an array variable, and p is a procedure name.

Procedures:

$$P \quad := \quad \text{procedure } p(v_1, \dots, v_n) \hat{=} S$$

where $v_1, \dots, v_n$ are variable names, p is a procedure name, and S is a program statement

Programs



THE UNIVERSITY OF
MELBOURNE

procedure $p_1(v_1, \dots, v_m) \hat{=} S_1$

...

procedure $p_n(v_1, \dots, v_n) \hat{=} S_n$

S

Programs



THE UNIVERSITY OF
MELBOURNE

procedure $p_1(v_1, \dots, v_m) \hat{=} S_1$

...

procedure $p_n(v_1, \dots, v_n) \hat{=} S_n$

S

“main” program:

Programs



THE UNIVERSITY OF
MELBOURNE

procedure $p_1(v_1, \dots, v_m) \hat{=} S_1$

...

procedure $p_n(v_1, \dots, v_n) \hat{=} S_n$

S

procedure FACTORIAL(**in** n , **out** f

$f := 1$

$i := 0$

while $i \neq n$ **do**

$i := i + 1$

$f := f \times i$

end while

end procedure

FACTORIAL(10, f)

“main” program:

Programs



procedure $p_1(v_1, \dots, v_m) \hat{=} S_1$

...

procedure $p_n(v_1, \dots, v_n) \hat{=} S_n$

S

procedure FACTORIAL(**in** n , **out** f

$f := 1$

$i := 0$

while $i \neq n$ **do**

$i := i + 1$

$f := f \times i$

end while

end procedure

FACTORIAL(10, f)

“main” program:

Proof Structure



THE UNIVERSITY OF
MELBOURNE

Follows the program structure

```
procedure FACTORIAL(in  $n$ , out  $f$   
     $f := 1$   
     $i := 0$   
    while  $i \neq n$  do  
         $i := i + 1$   
         $f := f \times i$   
    end while  
end procedure  
FACTORIAL(10,  $f$ )
```

To prove something about X ,
we reason about X 's internal components

Hoare Triples



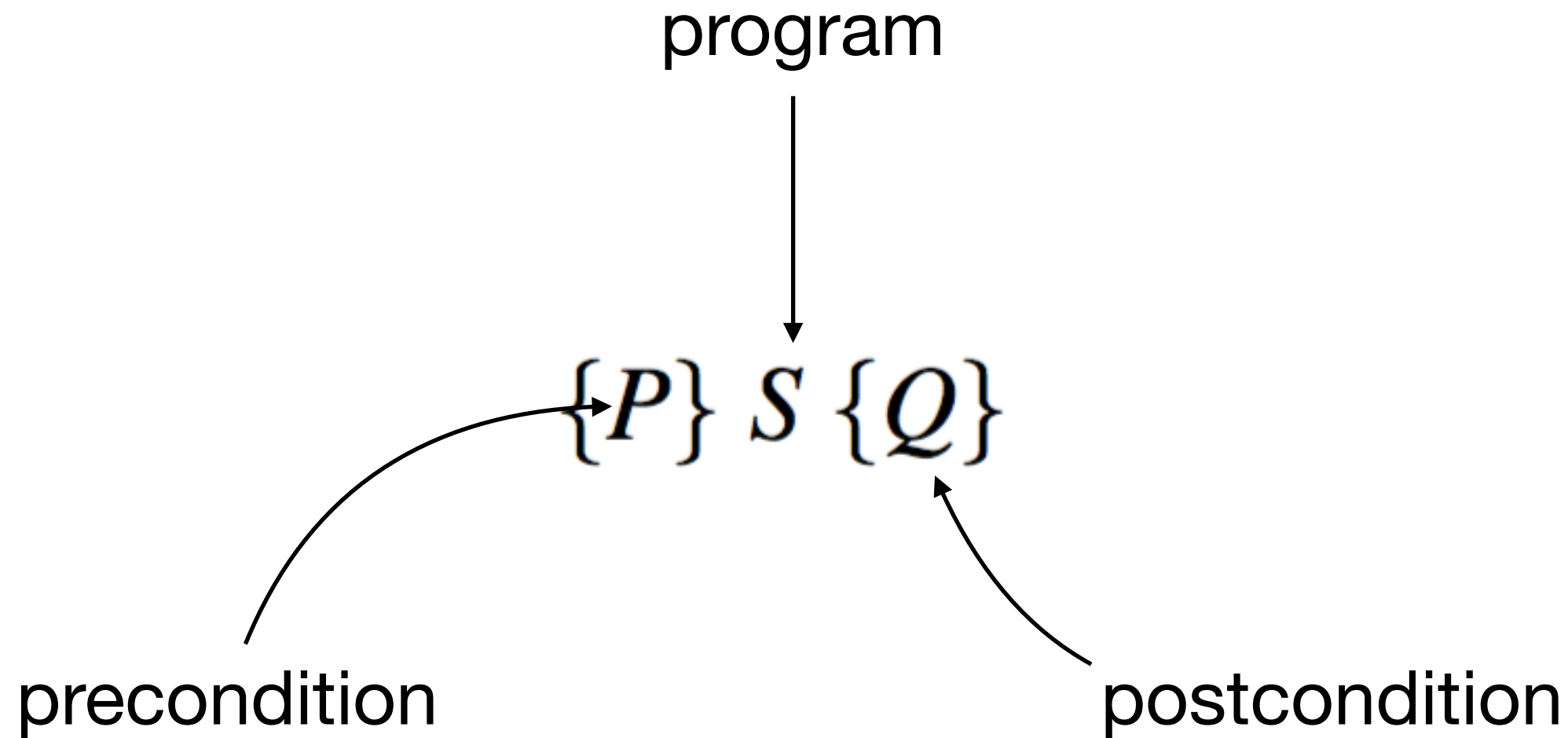
THE UNIVERSITY OF
MELBOURNE

$$\{P\} S \{Q\}$$

Hoare Triples



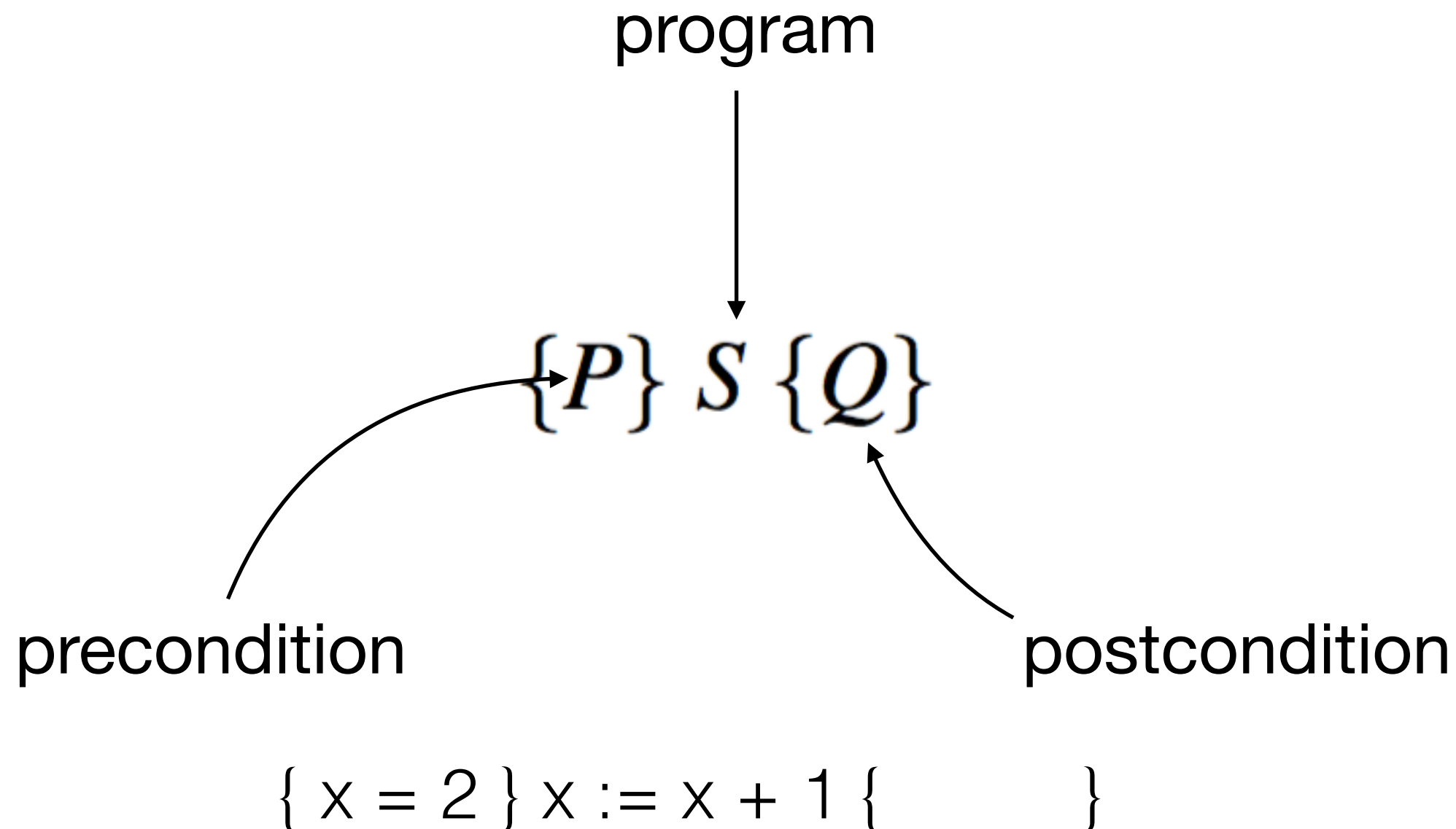
THE UNIVERSITY OF
MELBOURNE



Hoare Triples



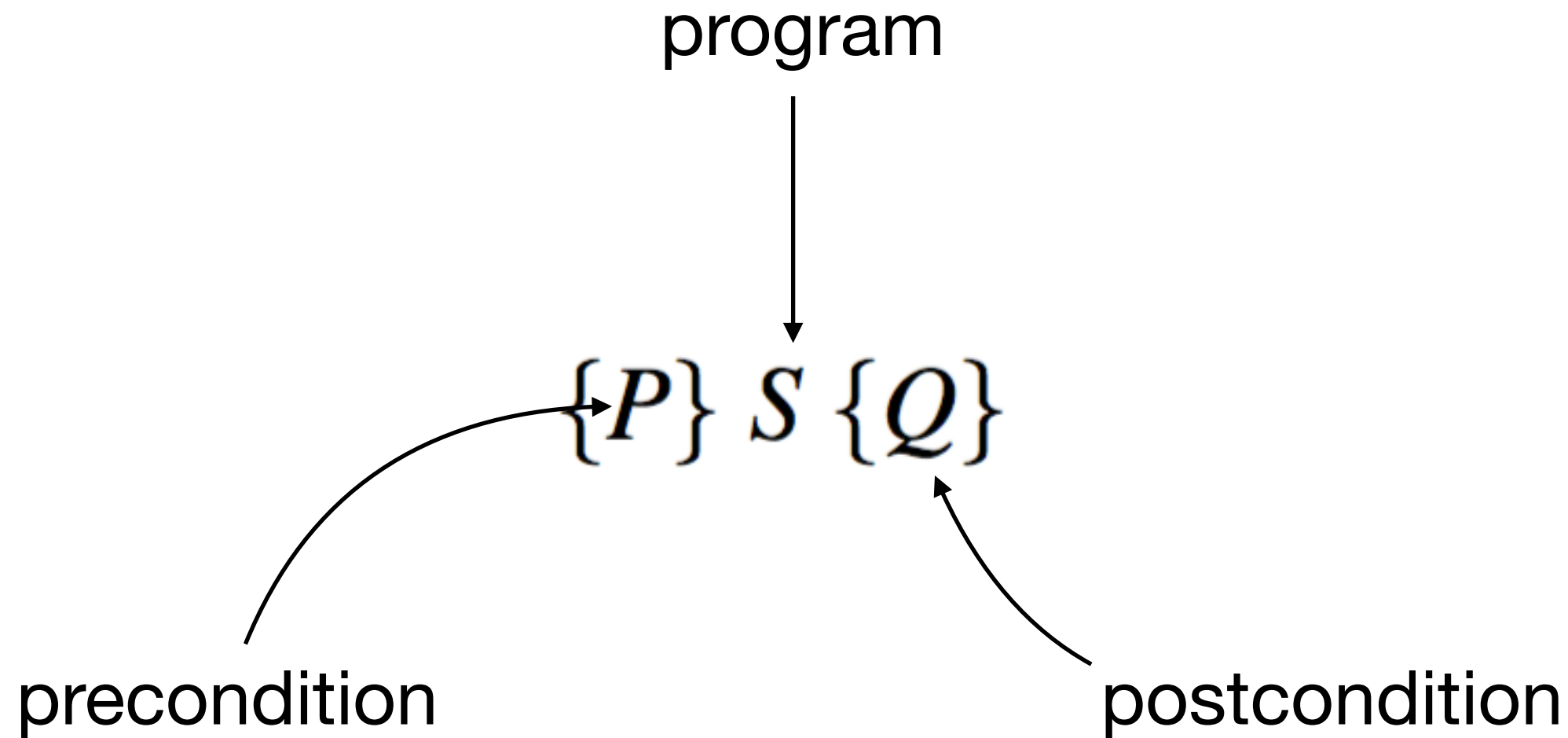
THE UNIVERSITY OF
MELBOURNE



Hoare Triples

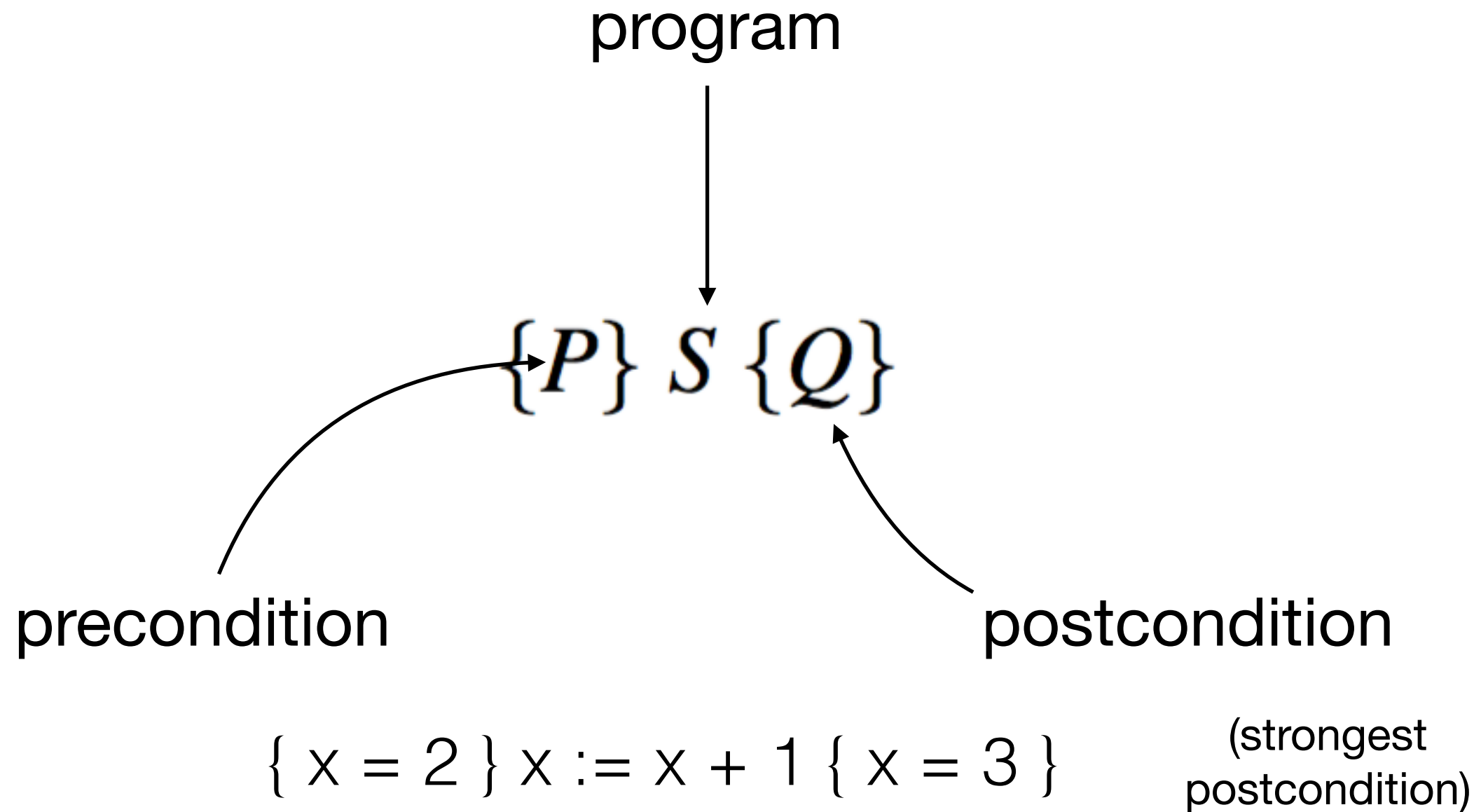


THE UNIVERSITY OF
MELBOURNE

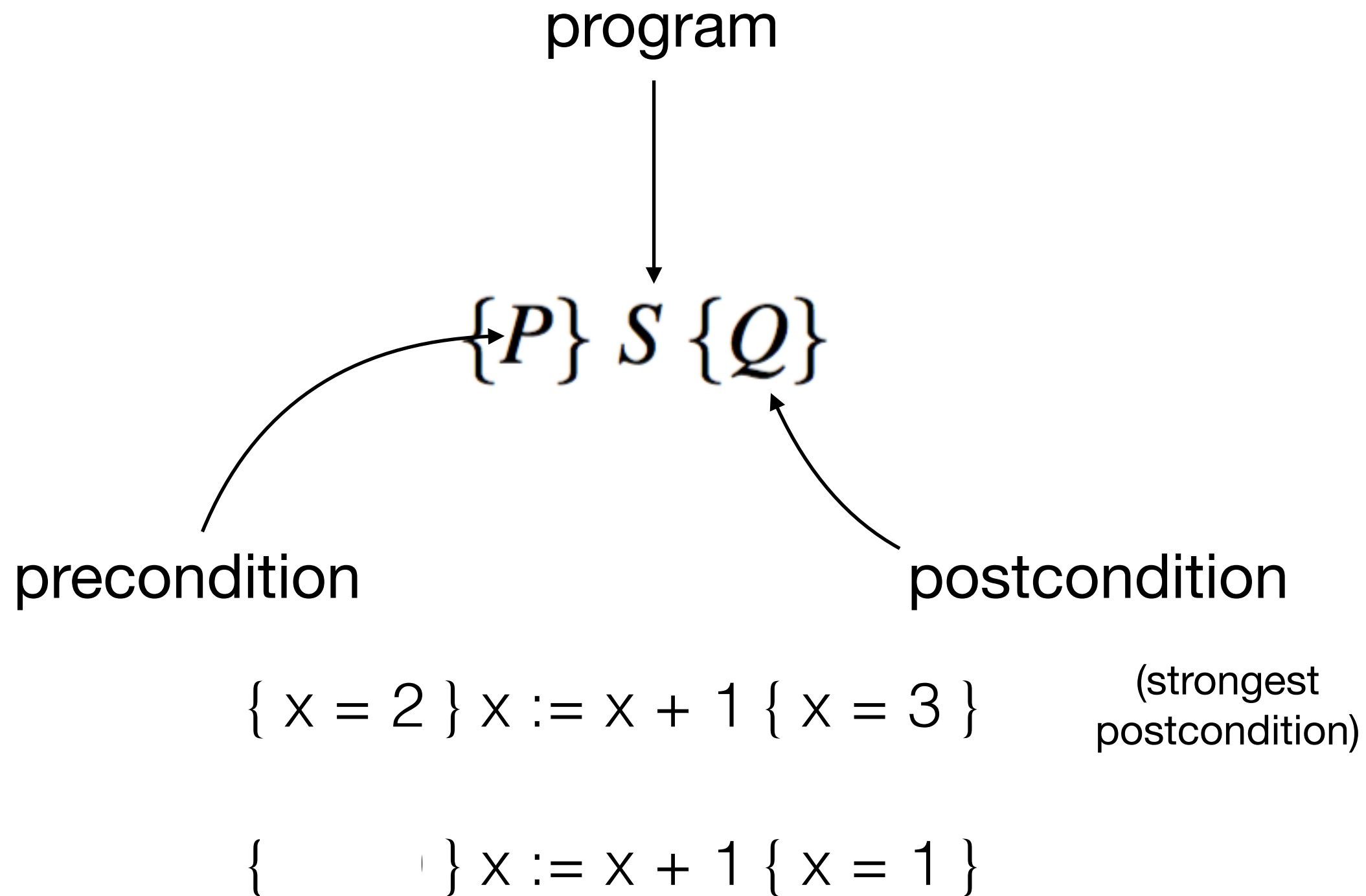


$\{ x = 2 \} x := x + 1 \{ x = 3 \}$

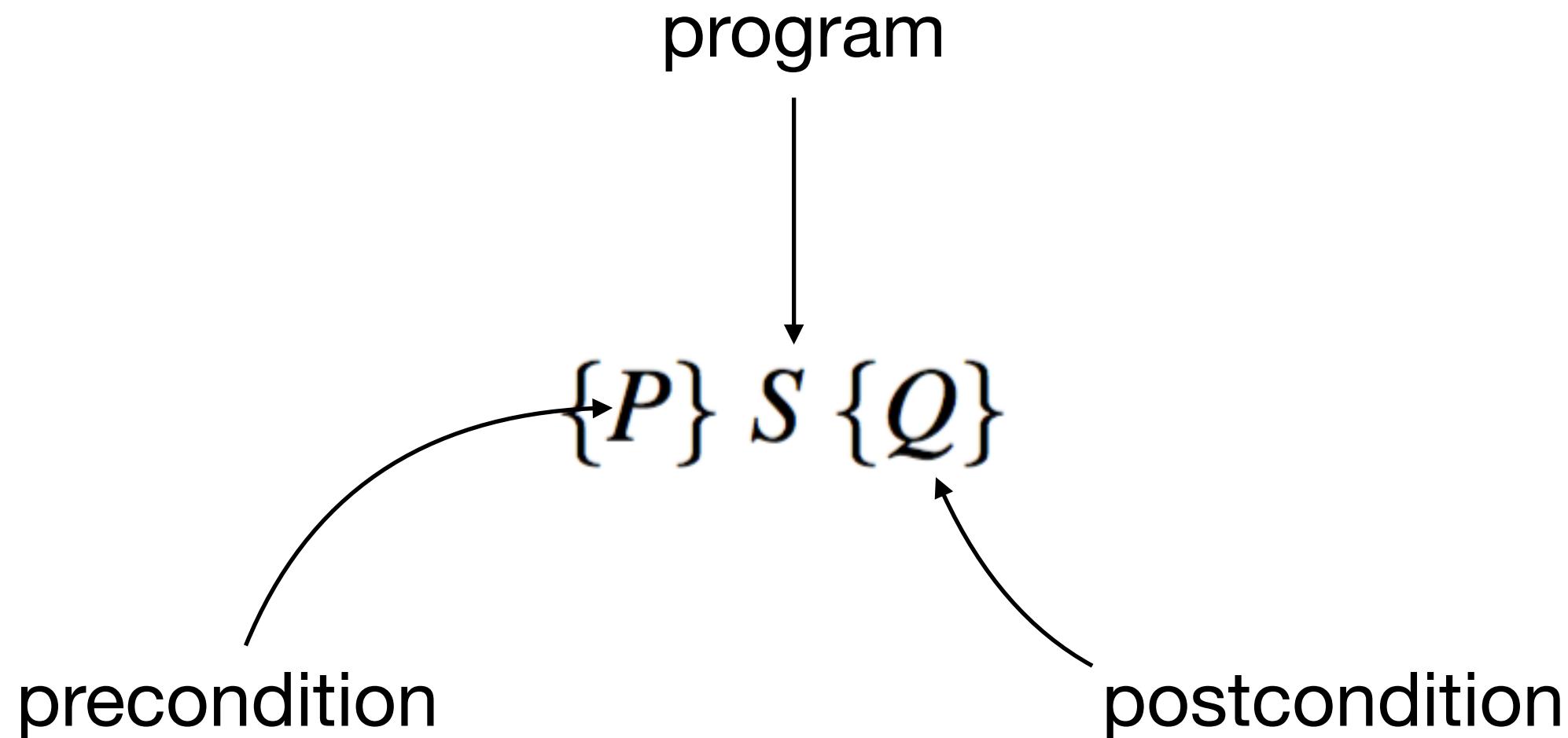
Hoare Triples



Hoare Triples



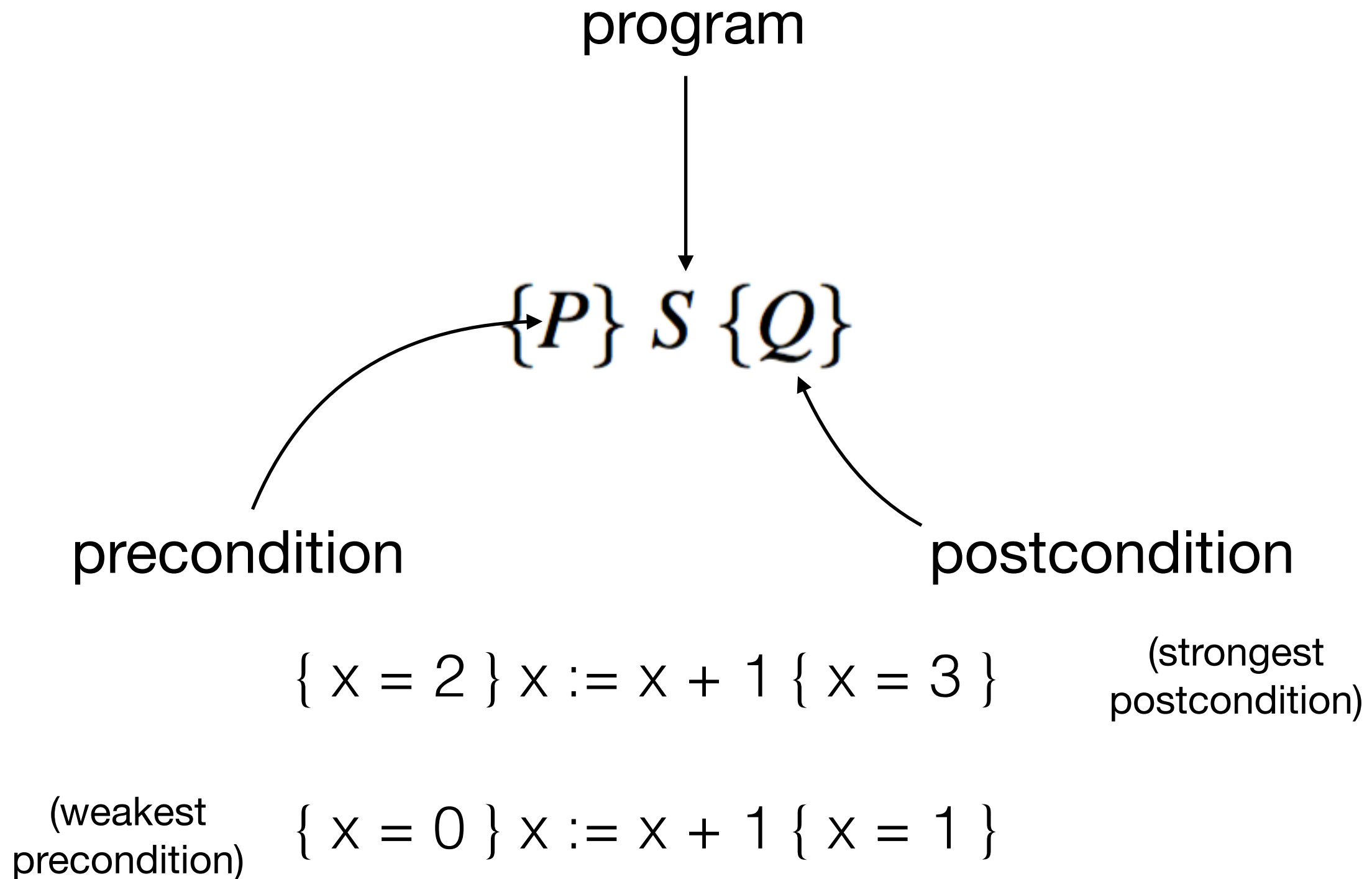
Hoare Triples



$\{x = 2\} x := x + 1 \{x = 3\}$ (strongest postcondition)

$\{x = 0\} x := x + 1 \{x = 1\}$

Hoare Triples



Logic (Refresher)

We use standard propositional symbols:

$A \Rightarrow B$	“implies”	if A is true then B is also true
-------------------	-----------	----------------------------------

$\neg A$	“not”, “negation”	A is not true
----------	----------------------	---------------

$A \wedge B$	“and”, “conjunction”	A is true and B is true
--------------	-------------------------	-------------------------

$A \vee B$	“or”, “disjunction”	at least one of A or B is true
------------	------------------------	--------------------------------

<i>true</i>		the proposition that is always true
-------------	--	-------------------------------------

<i>false</i>		the proposition that is never true
--------------	--	------------------------------------

Inference Rules



THE UNIVERSITY OF
MELBOURNE

Some trivial examples:

$$\frac{A \quad A \Rightarrow B}{B} \text{ *modus ponens* }$$

An axiom: $\frac{}{\text{true}}$

A structural rule: $\frac{A \quad B}{A \wedge B}$

Assignment Statements



THE UNIVERSITY OF
MELBOURNE

Assignment Statements



THE UNIVERSITY OF
MELBOURNE

$$\{ x = 2 \} x := x + 1 \{ \quad \}$$

Assignment Statements



THE UNIVERSITY OF
MELBOURNE

$$\{ x = 2 \} x := x + 1 \{ x = 3 \}$$

Assignment Statements



THE UNIVERSITY OF
MELBOURNE

$$\{ x = 2 \} x := x + 1 \{ x = 3 \}$$
$$\{ x \geq 0 \} x := x + 1 \{ \quad \quad \quad \}$$

Assignment Statements



THE UNIVERSITY OF
MELBOURNE

$$\{ x = 2 \} x := x + 1 \{ x = 3 \}$$

$$\{ x \geq 0 \} x := x + 1 \{ x - 1 \geq 0 \}$$

Assignment Statements



THE UNIVERSITY OF
MELBOURNE

$$\{ x = 2 \} x := x + 1 \{ x = 3 \}$$

$$\{ x \geq 0 \} x := x + 1 \{ x - 1 \geq 0 \}$$

$$\{ x \geq 0 \} x := \text{sha256}(x) \{ \quad \}$$

Assignment Statements



THE UNIVERSITY OF
MELBOURNE

$$\{ x = 2 \} x := x + 1 \{ x = 3 \}$$

$$\{ x \geq 0 \} x := x + 1 \{ x - 1 \geq 0 \}$$

$$\{ x \geq 0 \} x := \text{sha256}(x) \{ \quad ? \quad \}$$

Assignment Statements



THE UNIVERSITY OF
MELBOURNE

$$\{ x = 2 \} x := x + 1 \{ x = 3 \}$$

$$\{ x \geq 0 \} x := x + 1 \{ x - 1 \geq 0 \}$$

$$\{ x \geq 0 \} x := \text{sha256}(x) \{ \quad ? \quad \}$$

$$\{ \quad \quad \} x := x + 1 \{ x = 1 \}$$

Assignment Statements



THE UNIVERSITY OF
MELBOURNE

$$\{ x = 2 \} x := x + 1 \{ x = 3 \}$$

$$\{ x \geq 0 \} x := x + 1 \{ x - 1 \geq 0 \}$$

$$\{ x \geq 0 \} x := \text{sha256}(x) \{ \quad ? \quad \}$$

$$\{ x = 0 \} x := x + 1 \{ x = 1 \}$$

Assignment Statements



THE UNIVERSITY OF
MELBOURNE

$$\{ x = 2 \} x := x + 1 \{ x = 3 \}$$

$$\{ x \geq 0 \} x := x + 1 \{ x - 1 \geq 0 \}$$

$$\{ x \geq 0 \} x := \text{sha256}(x) \{ \quad ? \quad \}$$

$$\{ x = 0 \} x := x + 1 \{ x = 1 \}$$

$$\{ \quad \quad \quad \} x := x + 1 \{ x = 1 \}$$

Assignment Statements



THE UNIVERSITY OF
MELBOURNE

$$\{ x = 2 \} x := x + 1 \{ x = 3 \}$$

$$\{ x \geq 0 \} x := x + 1 \{ x - 1 \geq 0 \}$$

$$\{ x \geq 0 \} x := \text{sha256}(x) \{ \quad ? \quad \}$$

$$\{ x = 0 \} x := x + 1 \{ x = 1 \}$$

$$\{ x + 1 = 1 \} x := x + 1 \{ x = 1 \}$$

Assignment Statements



THE UNIVERSITY OF
MELBOURNE

$$\{ x = 2 \} x := x + 1 \{ x = 3 \}$$

$$\{ x \geq 0 \} x := x + 1 \{ x - 1 \geq 0 \}$$

$$\{ x \geq 0 \} x := \text{sha256}(x) \{ \quad ? \quad \}$$

$$\{ x = 0 \} x := x + 1 \{ x = 1 \}$$

$$\{ x + 1 = 1 \} x := x + 1 \{ x = 1 \}$$

program is adding 1 to x, and the result is $x = 1$ afterwards,
then it better be the case that $(\text{the old value of } x) + 1 = 1$ beforehand

Assignment Statements



THE UNIVERSITY OF
MELBOURNE

$$\{ x = 2 \} x := x + 1 \{ x = 3 \}$$

$$\{ x \geq 0 \} x := x + 1 \{ x - 1 \geq 0 \}$$

$$\{ x \geq 0 \} x := \text{sha256}(x) \{ \quad ? \quad \}$$

$$\{ x = 0 \} x := x + 1 \{ x = 1 \}$$

$$\{ x + 1 = 1 \} x := x + 1 \{ x = 1 \}$$

program is adding 1 to x, and the result is $x = 1$ afterwards,
then it better be the case that $(\text{the old value of } x) + 1 = 1$ beforehand

$$\{ \quad \quad \} x := x + 1 \{ x \geq 0 \}$$

Assignment Statements



THE UNIVERSITY OF
MELBOURNE

$$\{ x = 2 \} x := x + 1 \{ x = 3 \}$$

$$\{ x \geq 0 \} x := x + 1 \{ x - 1 \geq 0 \}$$

$$\{ x \geq 0 \} x := \text{sha256}(x) \{ \quad ? \quad \}$$

$$\{ x = 0 \} x := x + 1 \{ x = 1 \}$$

$$\{ x + 1 = 1 \} x := x + 1 \{ x = 1 \}$$

program is adding 1 to x, and the result is $x = 1$ afterwards,
then it better be the case that $(\text{the old value of } x) + 1 = 1$ beforehand

$$\{ x + 1 \geq 0 \} x := x + 1 \{ x \geq 0 \}$$

Assignment Statements



THE UNIVERSITY OF
MELBOURNE

$$\{ x = 2 \} x := x + 1 \{ x = 3 \}$$

$$\{ x \geq 0 \} x := x + 1 \{ x - 1 \geq 0 \}$$

$$\{ x \geq 0 \} x := \text{sha256}(x) \{ \quad ? \quad \}$$

$$\{ x = 0 \} x := x + 1 \{ x = 1 \}$$

$$\{ x + 1 = 1 \} x := x + 1 \{ x = 1 \}$$

program is adding 1 to x, and the result is $x = 1$ afterwards,
then it better be the case that $(\text{the old value of } x) + 1 = 1$ beforehand

$$\{ x + 1 \geq 0 \} x := x + 1 \{ x \geq 0 \}$$

$$\{ \quad \quad \quad \} x := \text{sha256}(x) \{ x \geq 0 \}$$

Assignment Statements



THE UNIVERSITY OF
MELBOURNE

$$\{ x = 2 \} x := x + 1 \{ x = 3 \}$$

$$\{ x \geq 0 \} x := x + 1 \{ x - 1 \geq 0 \}$$

$$\{ x \geq 0 \} x := \text{sha256}(x) \{ \quad ? \quad \}$$

$$\{ x = 0 \} x := x + 1 \{ x = 1 \}$$

$$\{ x + 1 = 1 \} x := x + 1 \{ x = 1 \}$$

program is adding 1 to x, and the result is $x = 1$ afterwards,
then it better be the case that $(\text{the old value of } x) + 1 = 1$ beforehand

$$\{ x + 1 \geq 0 \} x := x + 1 \{ x \geq 0 \}$$

$$\{ \text{sha256}(x) \geq 0 \} x := \text{sha256}(x) \{ x \geq 0 \}$$

Rule of Assignments



THE UNIVERSITY OF
MELBOURNE

$$\{P[E/x]\} x := E \{P\}$$

Rule of Assignments



THE UNIVERSITY OF
MELBOURNE

$$\{P[E/x]\} x := E \{P\}$$

$$\{ \quad \quad \quad \} x := x + 1 \{ x = 1 \}$$

Rule of Assignments



THE UNIVERSITY OF
MELBOURNE

$$\frac{}{\{P[E/x]\} x := E \{P\}}$$

$$\{ (x = 1)[x + 1/x] \} x := x + 1 \{ x = 1 \}$$

Rule of Assignments



THE UNIVERSITY OF
MELBOURNE

$$\frac{}{\{P[E/x]\} x := E \{P\}}$$

$$\{ (x = 1)[x + 1/x] \} x := x + 1 \{ x = 1 \}$$

$$\{ x + 1 = 1 \} x := x + 1 \{ x = 1 \}$$

Rule of Assignments



THE UNIVERSITY OF
MELBOURNE

$$\frac{}{\{P[E/x]\} x := E \{P\}}$$

$$\{ (x = 1)[x + 1/x] \} x := x + 1 \{ x = 1 \}$$

$$\{ x + 1 = 1 \} x := x + 1 \{ x = 1 \}$$

$$\{ x = 0 \} x := x + 1 \{ x = 1 \}$$

Rule of Assignments



THE UNIVERSITY OF
MELBOURNE

$$\frac{}{\{P[E/x]\} x := E \{P\}}$$

$$\{ (x = 1)[x + 1/x] \} x := x + 1 \{ x = 1 \}$$

$$\{ x + 1 = 1 \} x := x + 1 \{ x = 1 \}$$

$$\{ x = 0 \} x := x + 1 \{ x = 1 \}$$

$$\{ \quad \quad \quad \} x := x + 1 \{ x \geq 0 \}$$

Rule of Assignments



$$\frac{}{\{P[E/x]\} x := E \{P\}}$$

$$\{ (x = 1)[x + 1/x] \} x := x + 1 \{ x = 1 \}$$

$$\{ x + 1 = 1 \} x := x + 1 \{ x = 1 \}$$

$$\{ x = 0 \} x := x + 1 \{ x = 1 \}$$

$$\{ (x \geq 0)[x + 1/x] \} x := x + 1 \{ x \geq 0 \}$$

Rule of Assignments



$$\frac{}{\{P[E/x]\} x := E \{P\}}$$

$$\{ (x = 1)[x + 1/x] \} x := x + 1 \{ x = 1 \}$$

$$\{ x + 1 = 1 \} x := x + 1 \{ x = 1 \}$$

$$\{ x = 0 \} x := x + 1 \{ x = 1 \}$$

$$\{ (x \geq 0)[x + 1/x] \} x := x + 1 \{ x \geq 0 \}$$

$$\{ x + 1 \geq 0 \} x := x + 1 \{ x \geq 0 \}$$

Rule of Consequence



THE UNIVERSITY OF
MELBOURNE

Rule of Consequence



THE UNIVERSITY OF
MELBOURNE

$$\{ x = 0 \} x := x + 1 \{ x = 1 \}$$

Rule of Consequence



THE UNIVERSITY OF
MELBOURNE

$$\{ x = 0 \} x := x + 1 \{ x = 1 \}$$

$$\{ x = 0 \} x := x + 1 \{ x > 0 \}$$

Rule of Consequence



THE UNIVERSITY OF
MELBOURNE

$$\{ x = 0 \} x := x + 1 \{ x = 1 \}$$

$$\{ x = 0 \} x := x + 1 \{ x > 0 \}$$

$$\{ x \geq 0 \} x := x + 1 \{ x > 0 \}$$

Rule of Consequence



THE UNIVERSITY OF
MELBOURNE

$$\{ x = 0 \} x := x + 1 \{ x = 1 \}$$

$$\{ x = 0 \} x := x + 1 \{ x > 0 \}$$

$$\{ x \geq 0 \} x := x + 1 \{ x > 0 \}$$

$$\{ x = 0 \} x := x + 1 \{ x > 0 \}$$

Rule of Consequence



$$\{x = 0\} x := x + 1 \{x = 1\}$$

$$\{x = 0\} x := x + 1 \{x > 0\}$$

$$\{x \geq 0\} x := x + 1 \{x > 0\}$$

$$\{x = 0\} x := x + 1 \{x > 0\}$$

$$\frac{P' \Rightarrow P, Q \Rightarrow Q', \{P\} S \{Q\}}{\{P'\} S \{Q'\}}$$

Combining the Two Rules

$$\frac{}{\{P[E/x]\} \ x := E \ \{P\}}$$

Assignment axiom

$$\frac{P' \Rightarrow P, \ Q \Rightarrow Q', \ \{P\} \ S \ \{Q\}}{\{P'\} \ S \ \{Q'\}}$$

Consequence rule

Combining the Two Rules

$$\frac{}{\{P[E/x]\} \ x := E \ \{P\}}$$

Assignment axiom

$$\frac{P' \Rightarrow P, \ Q \Rightarrow Q', \ \{P\} \ S \ \{Q\}}{\{P'\} \ S \ \{Q'\}}$$

Consequence rule

Suppose we wish to prove:

Combining the Two Rules

$$\frac{}{\{P[E/x]\} x := E \{P\}}$$

Assignment axiom

$$\frac{P' \Rightarrow P, Q \Rightarrow Q', \{P\} S \{Q\}}{\{P'\} S \{Q'\}}$$

Consequence rule

Suppose we wish to prove:

$$\{x = 0\} x := x + 1 \{x > 0\}$$

Combining the Two Rules

$$\frac{}{\{P[E/x]\} \ x := E \ \{P\}} \qquad \frac{P' \Rightarrow P, \ Q \Rightarrow Q', \ \{P\} \ S \ \{Q\}}{\{P'\} \ S \ \{Q'\}}$$

Assignment axiom

Consequence rule

Apply consequence rule first

Suppose we wish to prove:

$$\{x = 0\} \ x := x + 1 \ \{x > 0\}$$

Combining the Two Rules



$$\{P[E/x]\} x := E \{P\}$$

Assignment axiom

$$\frac{P' \Rightarrow P, Q \Rightarrow Q', \{P\} S \{Q\}}{\{P'\} S \{Q'\}}$$

Consequence rule

$$\{x = 0\} x := x + 1 \{x > 0\}$$

Combining the Two Rules

$$\frac{}{\{P[E/x]\} x := E \{P\}}$$

Assignment axiom

$$\frac{P' \Rightarrow P, Q \Rightarrow Q', \{P\} S \{Q\}}{\{P'\} S \{Q'\}}$$

Consequence rule

$$\frac{\{?P\} x := x + 1 \{x > 0\}}{\{x = 0\} x := x + 1 \{x > 0\}}$$

Combining the Two Rules

$$\frac{}{\{P[E/x]\} x := E \{P\}}$$

Assignment axiom

$$\frac{P' \Rightarrow P, Q \Rightarrow Q', \{P\} S \{Q\}}{\{P'\} S \{Q'\}}$$

Consequence rule

$$\frac{x = 0 \Rightarrow ?P \quad \{ ?P \} x := x + 1 \{ x > 0 \}}{\{ x = 0 \} x := x + 1 \{ x > 0 \}}$$

Combining the Two Rules

$$\frac{}{\{P[E/x]\} \ x := E \ \{P\}} \qquad \frac{P' \Rightarrow P, \ Q \Rightarrow Q', \ \{P\} \ S \ \{Q\}}{\{P'\} \ S \ \{Q'\}}$$

Assignment axiom

Consequence rule

Now apply the assignment axiom

$$\frac{x = 0 \Rightarrow ?P \qquad \{ ?P \} \ x := x + 1 \ \{ x > 0 \}}{\{ x = 0 \} \ x := x + 1 \ \{ x > 0 \}}$$

Combining the Two Rules



$$\frac{}{\{P[E/x]\} x := E \{P\}}$$

Assignment axiom

$$\frac{P' \Rightarrow P, Q \Rightarrow Q', \{P\} S \{Q\}}{\{P'\} S \{Q'\}}$$

Consequence rule

$$x = 0 \Rightarrow x + 1 > 0 \quad \frac{}{\{x + 1 > 0\} x := x + 1 \{x > 0\}}$$

$$\{x = 0\} x := x + 1 \{x > 0\}$$

Combining the Two Rules

$$\frac{}{\{P[E/x]\} x := E \{P\}}$$

Assignment axiom

$$\frac{P' \Rightarrow P, Q \Rightarrow Q', \{P\} S \{Q\}}{\{P'\} S \{Q'\}}$$

Consequence rule

$$\frac{}{x = 0 \Rightarrow x + 1 > 0}$$

$$\frac{}{\{x + 1 > 0\} x := x + 1 \{x > 0\}}$$

$$\{x = 0\} x := x + 1 \{x > 0\}$$

Combining the Two Rules

$$\frac{}{\{P[E/x]\} x := E \{P\}} \qquad \frac{P' \Rightarrow P, Q \Rightarrow Q', \{P\} S \{Q\}}{\{P'\} S \{Q'\}}$$

Assignment axiom

Consequence rule

This is a general pattern:
apply consequence rule first to generalise precondition,
then apply other rules

$$\frac{x = 0 \Rightarrow x + 1 > 0 \quad \frac{}{\{x + 1 > 0\} x := x + 1 \{x > 0\}}}{\{x = 0\} x := x + 1 \{x > 0\}}$$

Combining the Two Rules

$$\frac{}{\{P[E/x]\} \ x := E \ \{P\}} \qquad \frac{P' \Rightarrow P, \ Q \Rightarrow Q', \ \{P\} \ S \ \{Q\}}{\{P'\} \ S \ \{Q'\}}$$

Assignment axiom

Consequence rule

This is a general pattern:
apply consequence rule first to generalise precondition,
then apply other rules

$$\frac{x = 0 \Rightarrow x + 1 > 0 \quad \{x + 1 > 0\} \ x := x + 1 \ \{x > 0\}}{\{x = 0\} \ x := x + 1 \ \{x > 0\}}$$